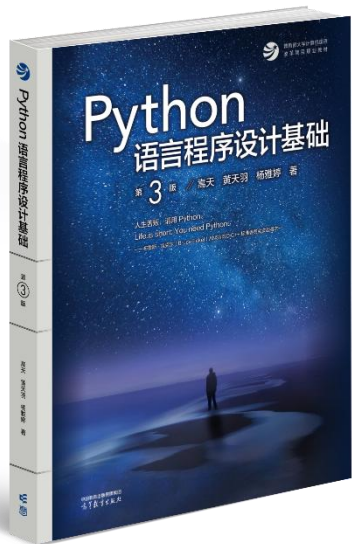


配套教材：高等教育出版社《Python语言程序设计基础（第3版）》

嵩天、黄天羽、杨雅婷 著



版权说明

- 本课程所有教学资料(课件)受CC BY-NC-SA 4.0知识产权协议保护
 - **允许**通过任何媒介复制传播及修改，未经授权**不能**商业使用
 - 非商业使用(如教学)**必须**以恰当且明确方式声明原作者信息
 - 若修改本资料并**再次**发布，**必须**遵守与本资料相同的许可条款
- 保护知识产权、分享发展成果、建立良性创新机制

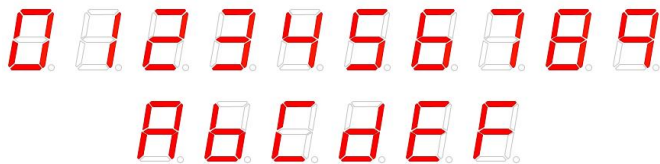
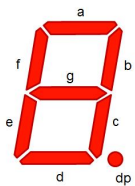
第5章 函数和代码复用

嵩 天
北京理工大学



本章概要

- 5.1 函数定义
- 5.2 函数调用
- 5.3 实例7: 七段数码管绘制
- 5.4 递归函数
- 5.5 代码复用和模块化设计
- 5.6 实例8: 科赫曲线绘制
- 5.7 PyInstaller库和程序打包



学习目标



- (1) 掌握函数的定义和调用方法
- (2) 理解函数的参数传递过程及变量的作用范围
- (3) 了解lambda()函数的用法
- (4) 掌握递归函数的定义和使用方法
- (5) 理解抽象、封装以及代码复用思想和模块化设计方法
- (6) 掌握常用的Python标准函数
- (7) 掌握PyInstaller库的使用方法，以及Python源文件的打包方法



5.1 函数定义

嵩 天
北京理工大学

什么是函数？



sum 、 len、 max、 min.....
abs、 round、

函数的分类

1. 内置函数: Python编译器自带的函数, 如input()、print()
2. 标准库函数: math库、random库、turtle库 (*import*)

import random

ls=list(random.randint(1,5))

3. 第三方库函数: pip install 安装

pip install numpy

#DayDayUpQ4.py

```
def dayUP(df):  
    dayup = 1.0  
    for i in range(1, 366):  
        if i % 7 in [6,0]:  
            dayup = dayup*(1 - 0.01)  
        else:  
            dayup = dayup*(1 + df)  
    return dayup
```

dayfactor = 0.01

while dayUP(dayfactor) < 37.78:

dayfactor += 0.001

print("每天需要努力: {:.3f}".format(dayfactor))

def..while..

("笨办法"试错)



函数的定义

函数是一段代码的表示

- 函数是一段具有特定功能的、可重用的语句组
- 函数是一种功能的抽象，一般函数表达特定功能
- 两个作用：降低编程难度 和 代码复用



函数的定义

函数是一段代码的表示

例子：

名称：咖啡机

输入：咖啡豆、水

处理：研磨、冲泡

输出：一杯咖啡

说明：同样的输入，每次都会得到相同的输出（如果机器工作正常）。咖啡机可以重复使用。

def <函数名>(<参数(0个或多个)>):

<函数体>

return <返回值>



函数的定义

计算 $n!$

函数名

参数

```
def fact(n) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    return s
```

返回值



函数的定义

$$y = f(x)$$

- 函数定义时，所指定的参数是一种**占位符**
- 函数定义后，如果不经**调用**，不会被执行
- 函数定义时，参数是输入、函数体是处理、结果是输出 (IPO)

参数个数



函数可以有参数，也可以没有，但必须保留括号

def <函数名>() :

<函数体>

return <返回值>

```
def f1() :  
    print("我也是函数")
```

```
def fact(n) :  
    s=1  
    for i in range(1,n+1):  
        s*=i  
    return s
```



可选参数

函数定义时可以为某些参数指定默认值，构成可选参数

def <函数名>(<非可选参数>, <可选参数>) :

<函数体>

return <返回值>



可选参数传递

计算 $n!//m$

可选参数

```
def fact(n, m=1) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    return s//m
```

```
>>> fact(10)
```

```
3628800
```

```
>>> fact(10,5)
```

```
725760
```




可变参数

函数定义时可以设计可变数量参数，既不确定参数总数量

```
def    <函数名>( <参数>,    *b    ) :
```

```
    <函数体>
```

```
    return    <返回值>
```



可变参数传递

计算 n! 乘数

可变参数

```
def fact(n, *b) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    for item in b:  
        s *= item  
    return s
```

```
>>> fact(10, 3)  
10886400
```

```
>>> fact(10, 3, 5, 8)  
435456000
```



函数的返回值

函数可以返回0个或多个结果

- *return* 保留字用来传递返回值
- 函数可以有返回值，也可以没有，可以有*return*，也可以没有
- *return* 可以传递0个返回值，也可以传递任意多个返回值



函数的返回值

函数调用时，参数可以按照位置或名称方式传递

```
def fact(n, m=1) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    return s//m, n, m
```

```
>>> fact( 10, 5 )
```

元组类型

```
(725760, 10, 5)
```

```
>>> a,b,c = fact(10,5)
```

```
>>> print(a,b,c)
```

```
725760 10 5
```



练习1：编写函数，求任意个连续整数的和

```
def calSum(n1,n2):  
    sum = 0  
    for i in range(n1, n2+1):  
        sum += i  
    print("sum=", sum)  
  
m1 = int(input("初值: "))  
m2 = int(input("终值: "))  
calSum(m1, m2)
```

练习2: 求 $(2+3+\dots+19+20)+(11+12+\dots+99+100)$ 的和



```
def calSum(n1,n2):
```

```
    sum = 0
```

```
    for i in range(n1, n2+1):
```

```
        sum += i
```

```
    return sum
```

```
print("sum=", calSum(2, 20) + calSum(11, 100))
```



练习3：思考如下三个问题：

- 1) 定义一个函数判断某个数是否是素数，是返回True，不是返回False
- 2) 找出2到100中所有的素数。
- 3) 找出2到100中所有的孪生素数。孪生素数就是指相差2的素数对，例如3和5，5和7，11和13等。



练习3: 1) 定义一个函数判断某个数是否是素数,

是返回True, 不是返回False

```
def prime(n):  
    for i in range(2, n):  
        if n%i == 0:  
            return False  
    else:  
        return True
```




练习3: 2) 找出2到100中所有的素数。

```
def prime(n):  
    for i in range(2, n):  
        if n%i == 0:  
            return False  
    else:  
        return True
```

```
for i in range(2, 100+1):  
    if prime(i) == True:  
        print("{:^4}".format(i), end='')
```



练习3: 3) 找出2到100中所有的孪生素数。孪生素数就是指相差2的素数对，例如3和5，5和7，11和13等。

```
def prime(n):  
    for i in range(2, n):  
        if n%i == 0:  
            return False  
    else:  
        return True  
  
for i in range(2,100+1):  
    if prime(i) == True and prime(i+2) == True:  
        print("{:^4},{:^4}".format(i,i+2))
```



lambda函数返回函数名作为结果

- lambda函数是一种匿名函数，即没有名字的功能
- 使用 *lambda* 保留字定义，函数名是返回结果
- lambda函数用于定义简单的、能够在一行内表示的函数

lambda函数



〈函数名〉 = *lambda* 〈参数〉: 〈表达式〉

def 〈函数名〉(〈参数〉) :

等价于

〈函数体〉

return 〈返回值〉

lambda函数



〈函数名〉 = *lambda* 〈参数〉: 〈表达式〉

```
>>> f = lambda x, y : x + y
```

```
>>> f(10, 15)
```

```
25
```

```
>>> f = lambda : "lambda函数"
```

```
>>> print(f())
```

```
lambda函数
```



lambda函数的应用

谨慎使用lambda函数

- lambda函数主要用作一些特定函数或方法的参数
- lambda函数有一些固定使用方式，建议逐步掌握
- 一般情况，建议使用`def`定义的普通函数



单元小结：函数定义

- 使用保留字`def`定义函数，`Lambda`定义匿名函数
- 可选参数(赋初值)、可变参数(*b)、名称传递
- 保留字`return`可以返回任意多个结果

5.2 函数调用



函数的调用

调用是运行函数代码的方式

```
def fact(n) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    return s
```

函数的定义

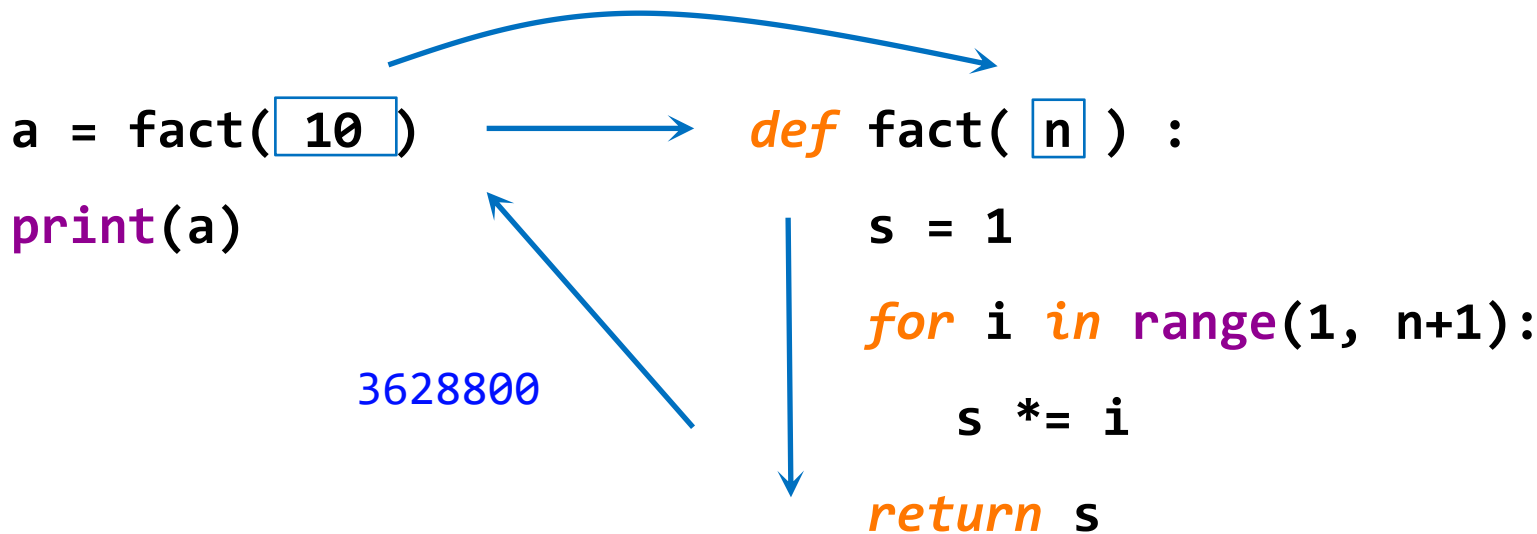
- 调用时要给出实际参数
- 实际参数替换定义中的参数
- 函数调用后得到返回值

fact(10) ←

函数的调用



函数的调用过程



参数传递的两种方式



位置传递：可以按照参数列表顺序提供实参

```
def fact(n, m=1) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    return s//m
```

```
>>> fact( 10, 5 )  
725760
```

位置传递

- 传递方式更直观
- 不适合参数较多的情况



参数传递的两种方式

名称传递：可以按照参数名字提供实参

```
def fact(n, m=1) :  
    s = 1  
    for i in range(1, n+1):  
        s *= i  
    return s//m
```

```
>>> fact( m=5, n=10 )  
725760
```

名称传递

- 可读性更好
- 参数顺序不影响正确性

局部变量和全局变量



程序
全局变量

＜语句块1＞

def ＜函数名＞(＜参数＞) :

＜函数体＞

return ＜返回值＞

＜语句块2＞

程序
局部变量



局部变量和全局变量

```
n, s = 10, 100
```



n和s是全局变量

```
def fact(n):
```

```
    s = 1
```



fact()函数中的n和s是局部变量

```
    for i in range(1, n+1):
```

```
        s *= i
```

```
    return s
```

```
print(fact(n), s)
```



n和s是全局变量

运行结果

```
>>>
```

3628800 100



局部变量和全局变量

规则1: 局部变量和全局变量是不同变量

- 局部变量是函数内部的占位符，与全局变量可能重名但不同
- 函数运算结束后，局部变量被释放
- 可以使用`global`保留字在函数内部使用全局变量



局部变量和全局变量

```
n, s = 10, 100
```

```
def fact(n) :
```

```
    s = 1
```

```
    for i in range(1, n+1):
```

```
        s *= i
```

```
    return s
```

```
print(fact(n), s)
```

fact()函数中s是局部变量
与全局变量s不同

运行结果

>>>

3628800 100

← 此处局部变量s是3628800

← 此处全局变量s是100



局部变量和全局变量

```
n, s = 10, 100
```

```
def fact(n) :
```

```
    global s
```

```
    for i in range(1, n+1):
```

```
        s *= i
```

```
    return s
```

```
print(fact(n), s)
```

fact()函数中使用global保留字声明

此处s是全局变量s

此处s指全局变量s

此处全局变量s被函数修改

运行结果

>>>

362880000 362880000

局部变量和全局变量



规则2: 局部变量为组合类型且未创建，等同于全局变量

`ls = ["F", "f"]` ← 通过使用[]真实创建了一个全局变量列表ls

`def func(a) :`

`ls.append(a)` ←

此处ls是列表类型，未真实创建
则等同于全局变量

`return`

`func("C")` ←

全局变量ls被修改

`print(ls)`

运行结果

`>>>`

`['F', 'f', 'C']`



局部变量和全局变量

```
ls = ["F", "f"]
```



通过使用[]真实创建了一个全局变量列表ls

```
def func(a) :
```

```
    ls = []
```



此处ls是列表类型，真实创建
ls是局部变量

```
    ls.append(a)
```

```
    return
```

```
func("C")
```



局部变量ls被修改

```
print(ls)
```

运行结果

```
>>>
```

```
['F', 'f']
```



局部变量和全局变量

使用规则

- 基本数据类型，无论是否重名，局部变量与全局变量不同
- 可以通过global保留字在函数内部声明全局变量
- 组合数据类型，如果局部变量未真实创建，则是全局变量



单元小结：函数调用

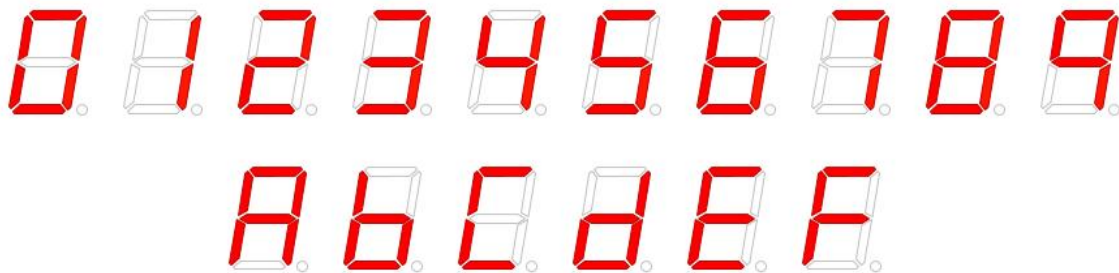
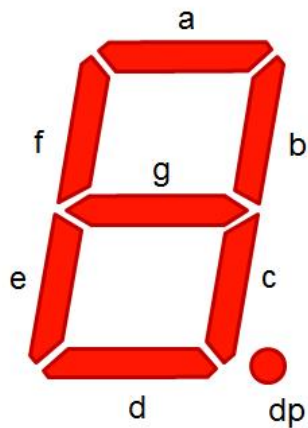
- 按位置顺序传递参数，按参数名称传递参数
- 局部变量和全局变量
- 保留字`global`声明使用全局变量

5.3 实例7: 七段数码管绘制

问题分析



七段数码管



七段数码管绘制

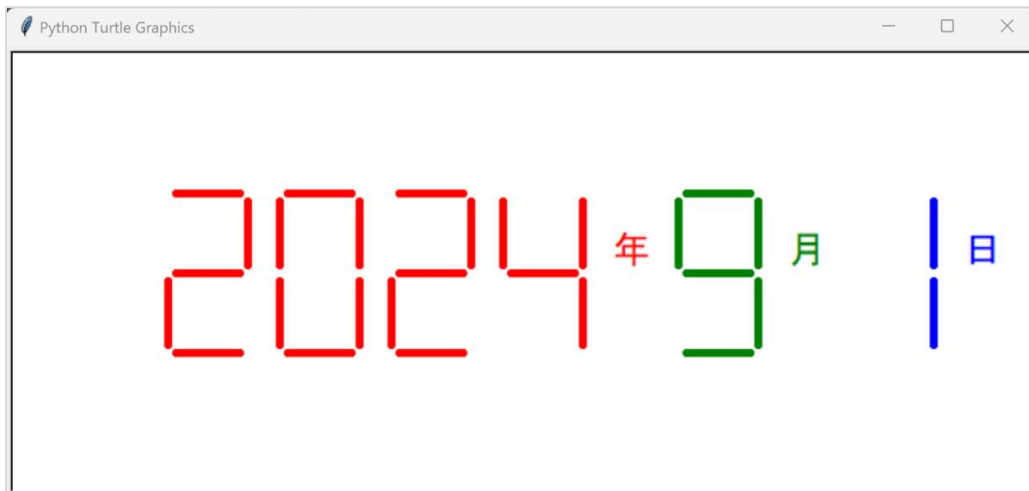
- 需求：用程序绘制七段数码管，似乎很有趣
- 该怎么做呢？

turtle绘图体系



七段数码管绘制

七段数码管绘制时间



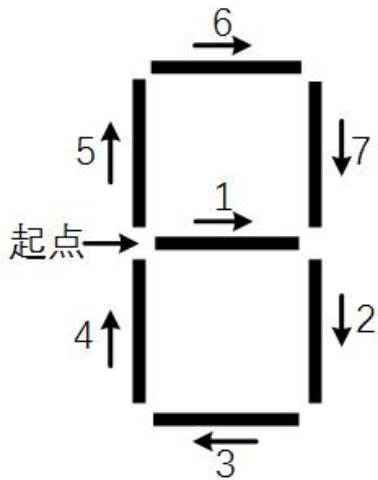


基本思路

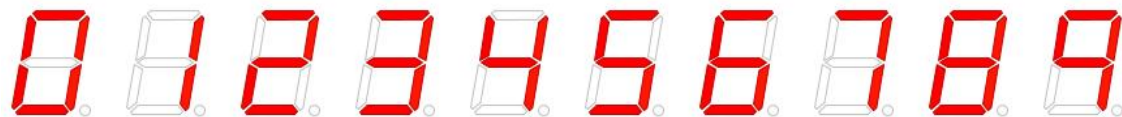
- 步骤1：绘制单个数字对应的数码管
- 步骤2：获得一串数字，绘制对应的数码管
- 步骤3：获得当前系统时间，绘制对应的数码管

七段数码管绘制

步骤1: 绘制单个数码管



- 七段数码管由7个基本线条组成
- 七段数码管可以有固定顺序
- 不同数字显示不同的线条



```
import turtle as t
```

```
def drawLine(draw): #绘制单段数码管
```

```
    t.pendown() if draw else t.penup()
```

```
    t.fd(40)
```

```
    t.right(90)
```

```
def drawDigit(digit): #根据数字绘制七段数码管
```

```
    drawLine(True) if digit in [2,3,4,5,6,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,1,3,4,5,6,7,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,2,3,5,6,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,2,6,8] else drawLine(False)
```

```
    t.left(90)
```

```
    drawLine(True) if digit in [0,4,5,6,8,9] else drawLine(False)
```

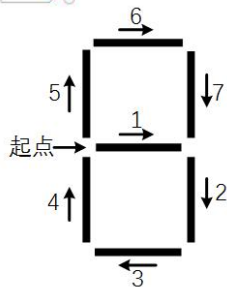
```
    drawLine(True) if digit in [0,2,3,5,6,7,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,1,2,3,4,7,8,9] else drawLine(False)
```

```
    t.left(180)
```

```
    t.penup() #为绘制后续数字确定位置
```

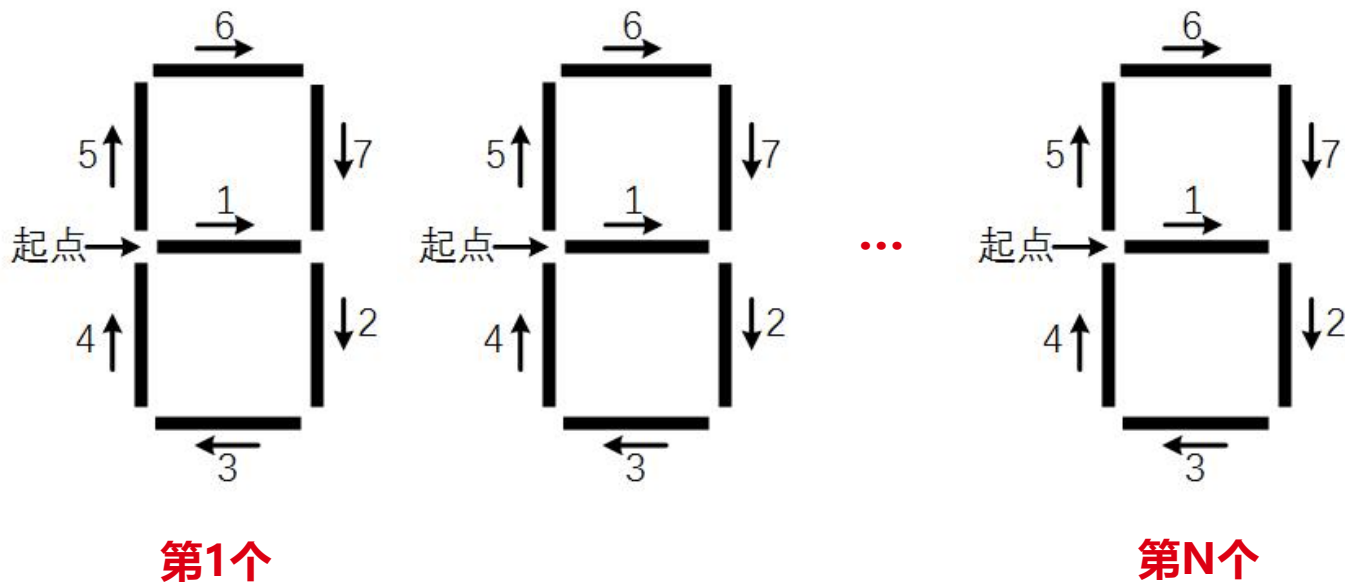
```
    t.fd(20) #为绘制后续数字确定位置
```



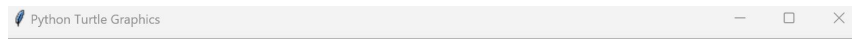
第段数码管绘制



步骤2: 获取一段数字，绘制多个数码管



```
import turtle as t
def drawLine(draw):    #绘制单段数码管
    ... (略)
def drawDigit(digit):  #根据数字绘制七段数码管
    ... (略)
def drawDate(date):    #获得要输出的数字
    for i in date:
        drawDigit(eval(i))    #通过eval()函数将数字变为整数
def main():
    t.setup(800, 350, 200, 200)
    t.penup()
    t.fd(-300)
    t.pensize(5)
    drawDate('20240922')
    t.hideturtle()
    t.done()
main()
```



20240922



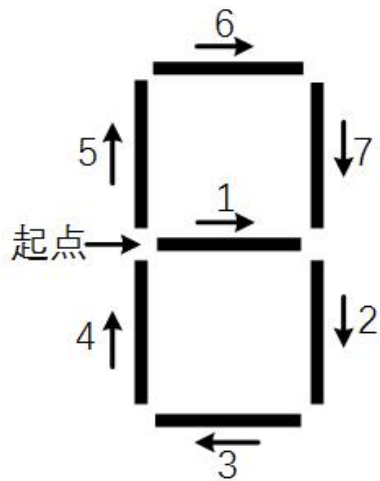
基本思路

- 步骤1：绘制单个数字对应的数码管
- 步骤2：获得一串数字，绘制对应的数码管
- 步骤3：获得当前系统时间，绘制对应的数码管

七段数码管绘制



绘制漂亮的七段数码管



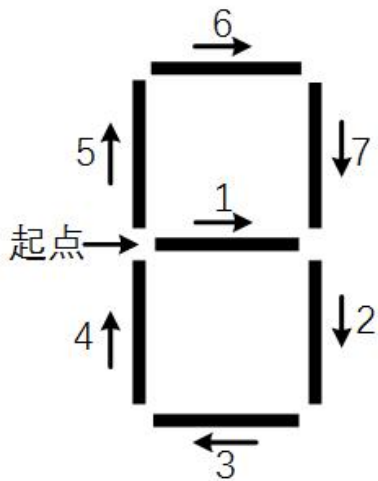
- 增加七段数码管之间线条间隔


```
import turtle as t
def drawGap():    #绘制数码管间隔
    t.penup()
    t.fd(5)
def drawLine(draw):    #绘制单段数码管
    drawGap()
    t.pendown() if draw else t.penup()
    t.fd(40)
    drawGap()
    t.right(90)
def drawDigit(digit): #根据数字绘制七段数码管
    drawLine(True) if digit in [2,3,4,5,6,8,9] else drawLine(False)
    drawLine(True) if digit in [0,1,3,4,5,6,7,8,9] else drawLine(False)
    drawLine(True) if digit in [0,2,3,5,6,8,9] else drawLine(False)
    drawLine(True) if digit in [0,2,6,8] else drawLine(False)
    ... (略)
```

七段数码管绘制



步骤3: 获取系统时间，绘制七段数码管



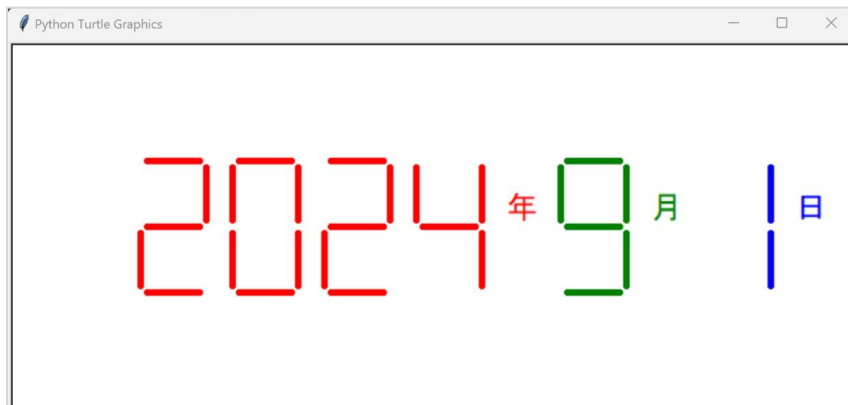
- 使用time库获得系统当前时间
- 增加年月日标记
- 年月日颜色不同

```

import turtle as t
import time
...(略)
def drawDate(date):    #data为日期, 格式为 '%Y-%m=%d+'
    t.pencolor("red")
    for i in date:
        if i == '-':
            t.write('年',font=("Arial", 18, "normal"))
            t.pencolor("green")
            t.fd(40)
        elif i == '=':
            t.write('月',font=("Arial", 18, "normal"))
            t.pencolor("blue")
            t.fd(40)
        elif i == '+':
            t.write('日',font=("Arial", 18, "normal"))
        else:
            drawDigit(eval(i))
def main():
...(略)

```

```
import turtle as t
import time
...(略)
def drawDate(date):
...(略)
def main():
    t.setup(800, 350, 200, 200)
    t.penup()
    t.fd(-300)
    t.pensize(5)
    drawDate(time.strftime('%Y-%m=%d+', time.gmtime()))
    t.hideturtle()
    t.done()
main()
```



```
import turtle as t
```

```
import time
```

```
...(略)
```

```
def drawLine(draw):
```

```
    drawGap()
```

```
    t.pendown() if draw else t.penup()
```

```
    t.fd(40)
```

```
    drawGap()
```

```
    t.right(90)
```

```
def drawDigit(digit):
```

```
    drawLine(True) if digit in [2,3,4,5,6,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,1,3,4,5,6,7,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,2,3,5,6,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,2,6,8] else drawLine(False)
```

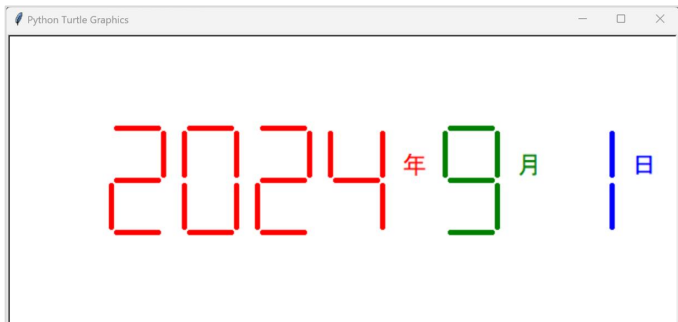
```
    t.left(90)
```

```
    drawLine(True) if digit in [0,4,5,6,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,2,3,5,6,7,8,9] else drawLine(False)
```

```
    drawLine(True) if digit in [0,1,2,3,4,7,8,9] else drawLine(False)
```

```
...(略)
```





理解方法思维

- 模块化思维：确定模块接口，封装功能
- 规则化思维：抽象过程为规则，计算机自动执行
- 化繁为简：将大功能变为小功能组合，分而治之

应用问题的扩展

- 绘制带小数点的七段数码管
- 带刷新的时间倒计时效果
- 绘制高级的数码管



5.4 递归函数

递归的定义



函数定义中调用函数自身的方式

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & \text{otherwise} \end{cases}$$



递归的定义

两个关键特征

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & otherwise \end{cases}$$

- 递归链条：计算过程存在递归链条
- 递归基例：存在一个或多个不需要再次递归的基例



递归的定义

类似数学归纳法

- 数学归纳法
 - 证明当 n 取第一个值 n_0 时命题成立
 - 假设当 n_k 时命题成立，证明当 $n=n_{k+1}$ 时命题也成立
- 递归是数学归纳法思维的编程体现



递归的实现

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & \text{otherwise} \end{cases}$$

```
def fact(n):  
    if n == 0 :  
        return 1  
    else :  
        return n*fact(n-1)
```

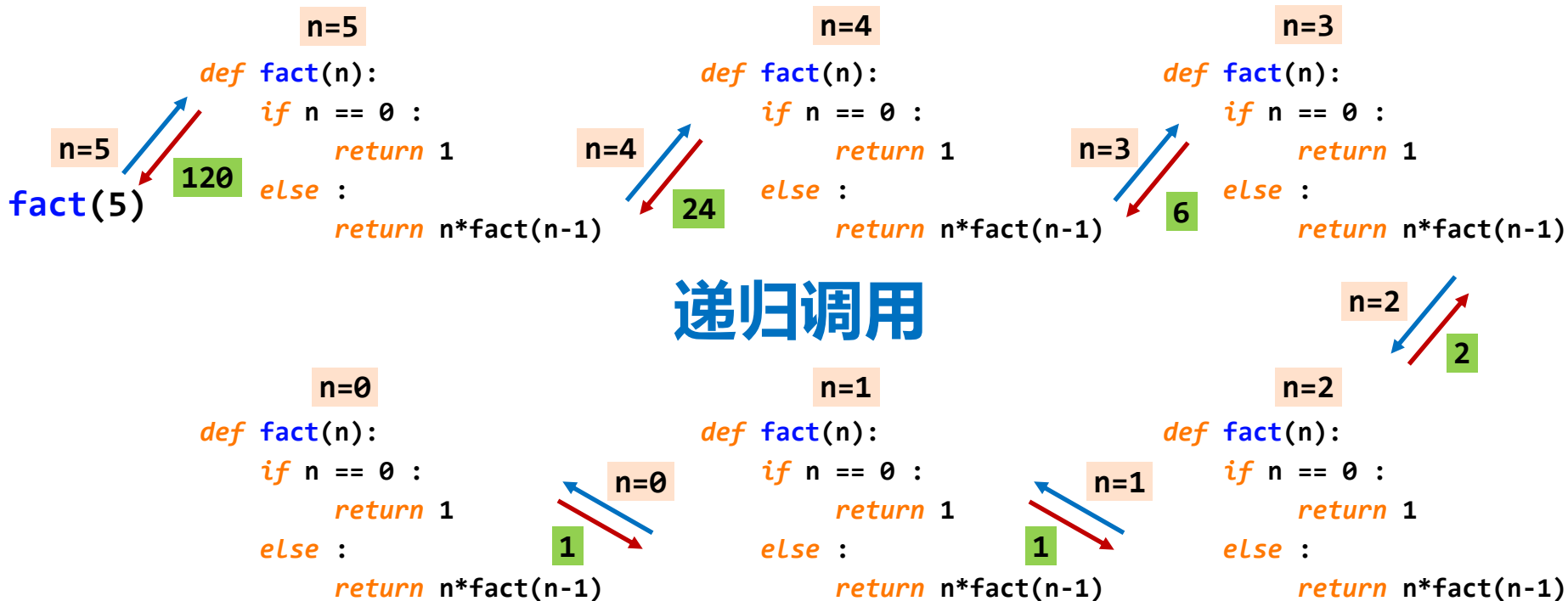


函数 + 分支语句

- 递归本身是一个函数，需要函数定义方式描述
- 函数内部，采用分支语句对输入参数进行判断
- 基例和链条，分别编写对应代码



递归的调用过程





字符串反转

将字符串s反转后输出

思路分析:

使用递归实现字符串反转的关键在于将问题分解为更小的子问题。

- **递归基例**: 当字符串为空 ($s == ""$) , 直接返回该字符串 (因为反转后不变) 。
- **递归链条**: 对于字符串s, 将其分为第一个字符s[0]和剩余部分s[1:]。然后递归调用函数对剩余部分进行反转, 并将第一个字符追加到反转后的剩余字符串的末尾。即: $rvs(s[1:]) + s[0]$ 。

字符串反转



将字符串s反转后输出

```
>>> s[::-1]
```

- 函数 + 分支结构

```
def rvs(s):
```

```
    if s == "" :
```

```
        return s
```

```
    else :
```

```
        return rvs(s[1:])+s[0]
```

- 递归链条

- 递归基例

一个经典数列

$$F(n) = \begin{cases} 1 & n = 1 \\ 1 & n = 2 \\ F(n-1) + F(n-2) & \textit{otherwise} \end{cases}$$

斐波那契数列



$$F(n) = F(n-1) + F(n-2)$$

- 函数 + 分支结构

- 递归链条

- 递归基例

```
def f(n):  
    if n == 1 or n == 2 :  
        return 1  
    else :  
        return f(n-1) + f(n-2)
```



递归练习

练习1：编写递归函数计算一个数字各位数字之和

```
def digit_sum(n):  
    if n < 10:  
        return n  
    else:  
        return n % 10 + digit_sum(n // 10)
```

```
n=eval(input("digit="))  
print(digit_sum(n))
```



递归练习

练习2：编写一个递归函数，计算下面的级数：

$$m(i)=1+1/2+1/3+.....1/i$$

```
def fun(n):  
    if n == 1:  
        return 1  
    else:  
        return fun(n - 1) + 1 / n  
  
print(fun(1))  
print(fun(2))  
print(fun(10))  
print(fun(100))  
print(fun(500))|
```

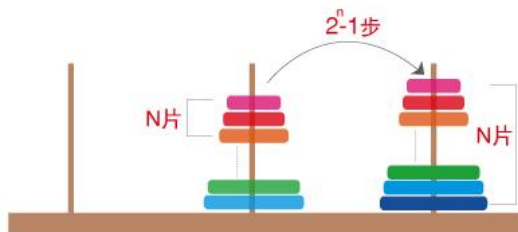
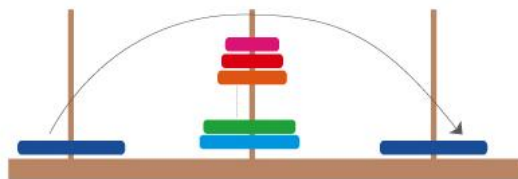
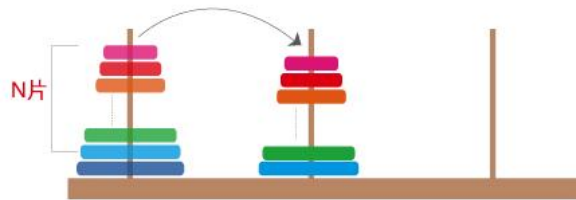
汉诺塔



目标：将所有圆盘从第一个柱子移动到第三个柱子上。

规则：

- ① 一次只能移动一个圆盘。
- ② 移动过程中，任何时候都不能将较大的圆盘放在较小的圆盘之上。
- ③ 可以借助第二个柱子作为辅助。





问题分析（递归思想）

递归的核心思想是：将复杂问题分解为结构相同但规模更小的子问题。

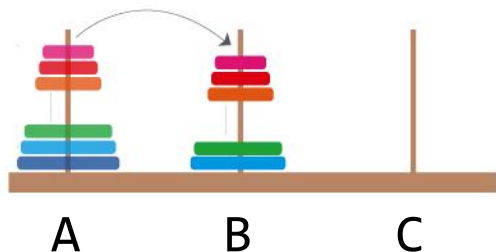
对于有 n 个圆盘的汉诺塔问题，我们可以这样分解：

第一步：将上面的 $n-1$ 个圆盘从 源柱子 移动到 辅助柱子（借助目标柱子）。

第二步：将最大的第 n 个圆盘从 源柱子 直接移动到 目标柱子。

第三步：将刚才那 $n-1$ 个圆盘从 辅助柱子 移动到 目标柱子（借助源柱子）。

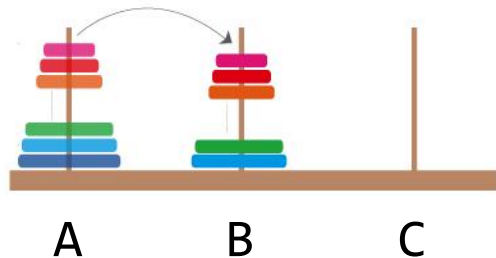
汉诺塔



- 函数 + 分支结构
- 递归链条
- 递归基例

```
count = 0
def hanoi(n, src, dst, mid):
    global count
    if n == 1 :
        print("{}: {}->{}".format(1,src,dst))
        count += 1
    else :
        hanoi(n-1, src, mid, dst)
        print("{}: {}->{}".format(n,src,dst))
        count += 1
        hanoi(n-1, mid, dst, src)
```

汉诺塔



```
count = 0
```

```
def hanoi(n, src, dst, mid):
```

```
    ... (略)
```

```
hanoi(3, "A", "C", "B")
```

```
print("总移动步数: ", count)
```

```
>>>
```

```
1:A->C
```

```
2:A->B
```

```
1:C->B
```

```
3:A->C
```

```
1:B->A
```

```
2:B->C
```

```
1:A->C
```

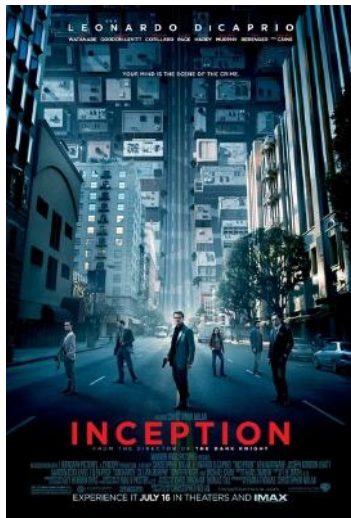
```
总移动步数: 7
```


如何理解递归呢？

递归很简单，无非就是一个函数调用自己而已...

- 看过《盗梦空间》吗？本质上就是递归...
- 学过数学归纳法吗？本质上就是递归...
- 听过这个故事吗？本质上就是递归...

"从前有座山，山里有座庙，庙里有个老和尚在讲故事..."





单元小结：递归函数

- 函数递归的2个特征：基例和链条
- 函数递归的实现：函数 + 分支结构
- 递归典型实例：字符串反转、斐波那契数列、汉诺塔

5.5 代码复用和模块化设计

嵩 天
北京理工大学



把代码当成资源进行抽象

- **代码资源化：** 程序代码是一种用来表达计算的“资源”
- **代码抽象化：** 使用函数等方法对代码赋予更高级别的定义
- **代码复用：** 同一份代码在需要时可以被重复使用



函数 和 对象 是代码复用的两种主要形式

函数：将代码命名
在代码层面建立了初步抽象

对象：属性和方法

`<a>.<b>` 和 `<a>.<b>()`

在函数之上再次组织进行抽象

抽象级别



分而治之

- 通过函数或对象封装将程序划分为模块及模块间的表达
- 具体包括：主程序、子程序和子程序间关系
- 分而治之：一种分而治之、分层抽象、体系化的设计思想



紧耦合 松耦合

- 紧耦合：两个部分之间交流很多，无法独立存在
- 松耦合：两个部分之间交流较少，可以独立存在
- 模块内部紧耦合、模块之间松耦合



单元小结：代码复用和模块化设计

- 通过抽象赋予代码含义：函数、对象
- 分而治之、分层抽象
- 模块化设计：松耦合、紧耦合

5.6 实例8: 科赫曲线绘制

嵩 天
北京理工大学

高大上的分形几何

- 分形几何是一种迭代的几何图形，广泛存在于自然界中



科赫雪花



科赫曲线，也叫雪花曲线



用Python绘制科赫曲线

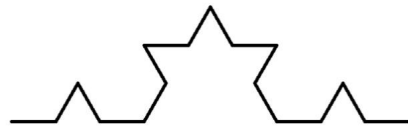
0阶科赫曲线



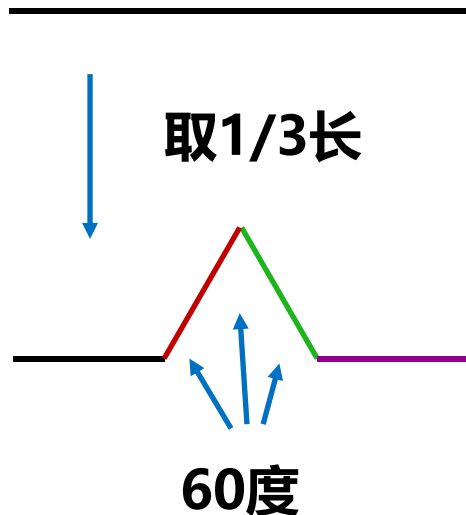
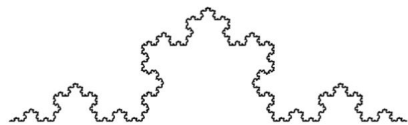
1阶科赫曲线



2阶科赫曲线

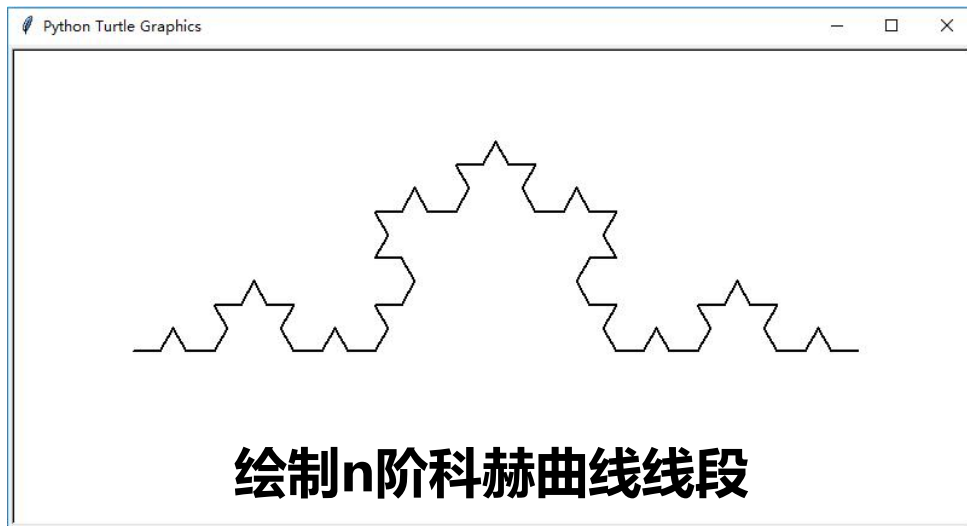


5阶科赫曲线



每分隔一次为一阶

科赫曲线的绘制



科赫曲线的绘制

#KochDrawV1.py

```
import turtle as t
```

```
def koch(size, n):
```

```
    if n == 0:
```

```
        t.fd(size)
```

```
    else:
```

```
        for angle in [0, 60, -120, 60]:
```

```
            t.left(angle)
```

```
            koch(size/3, n-1)
```

- 递归思想：函数+分支

- 递归链条：线段的组合

- 递归基例：初始线段

科赫曲线的绘制

```
#KochDrawV1.py
```

```
import turtle as t
```

```
def koch(size, n):
```

```
    if n == 0:
```

```
        t.fd(size)
```

```
    else:
```

```
        for angle in [0, 60, -120, 60]:
```

```
            t.left(angle)
```

```
            koch(size/3, n-1)
```

```
def main():
```

```
    rank = 3
```

```
    t.setup(800,400)
```

```
    t.speed(0)
```

```
    t.penup()
```

```
    t.goto(-300, -50)
```

```
    turtle.pendown()
```

```
    turtle.pensize(2)
```

```
    koch(600, rank)    # 绘制曲线
```

```
    turtle.hideturtle()
```

```
main()
```

科赫曲线绘制



```
#KochDrawV2.py
import turtle as t
def koch(size, n):
    ... (略)
```

```
def main():
    rank = 5
    t.setup(600, 600)
    t.speed(0)
    t.penup()
    t.goto(-200, 100)
    t.pendown()
    t.pensize(2)
    koch(400, rank)
    t.right(120)
    koch(400, rank)
    t.right(120)
    koch(400, rank)
    t.hideturtle()
main()
```

科赫曲线的绘制



科赫雪花的绘制

科赫曲线绘制



```
#KochDrawV2.py
```

```
import turtle as t
```

```
def koch(size, n):  
    ... (略)
```

```
def main():
```

```
    rank = 5
```

```
    t.setup(600,600)
```

```
    t.speed(0)
```

```
    t.penup()
```

```
    t.goto(-200, 100)
```

```
    t.pendown()
```

```
    t.pensize(2)
```

```
    koch(400, rank)
```

```
    t.right(120)
```

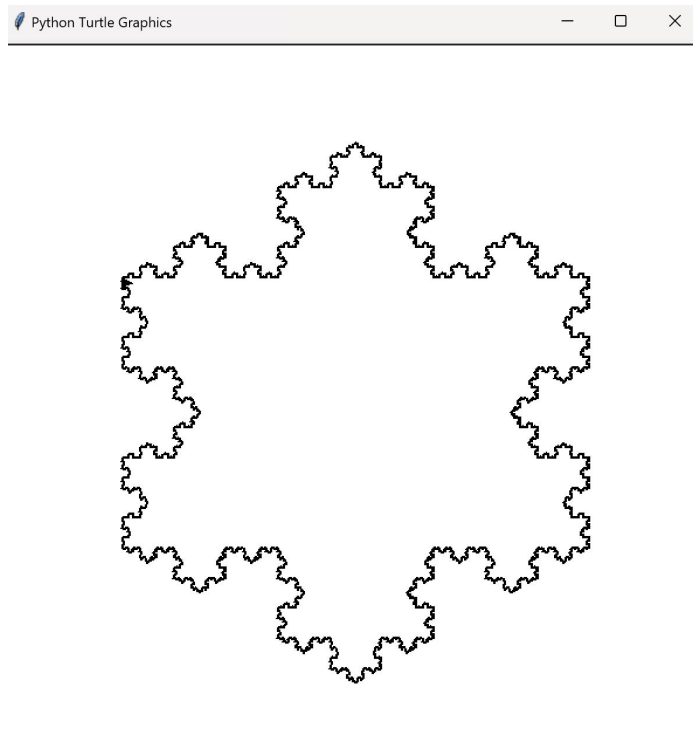
```
    koch(400, rank)
```

```
    t.right(120)
```

```
    koch(400, rank)
```

```
    t.hideturtle()
```

```
main()
```



科赫曲线绘制



```
#KochDrawV2.py
```

```
import turtle as t
```

```
def koch(size, n):
```

```
    if n == 0:
```

```
        t.fd(size)
```

```
    else:
```

```
        for angle in [0, 60, -120, 60]:
```

```
            t.left(angle)
```

```
            koch(size/3, n-1)
```

```
def main():
```

```
    rank = 5
```

```
    t.setup(600,600)
```

```
    t.speed(0)
```

```
    t.penup()
```

```
    t.goto(-200, 100)
```

```
    t.pendown()
```

```
    t.pensize(2)
```

```
    koch(400, rank)
```

```
    t.right(120)
```

```
    koch(400, rank)
```

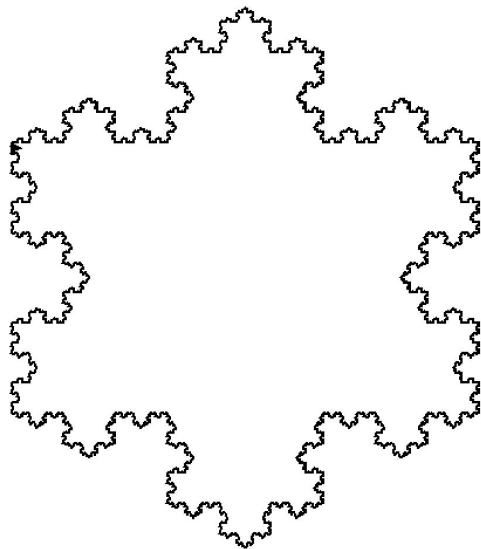
```
    t.right(120)
```

```
    koch(400, rank)
```

```
    t.hideturtle()
```

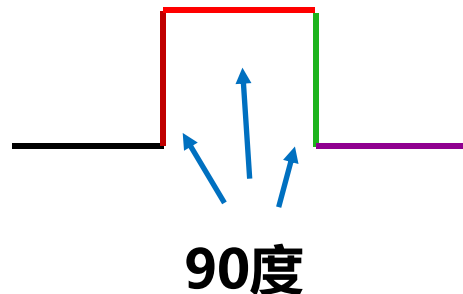
```
main()
```

Python Turtle Graphics



绘制条件的扩展

- 修改分形几何绘制阶数
- 修改科赫曲线的基本定义及旋转角度
- 修改绘制科赫雪花的基础框架图形





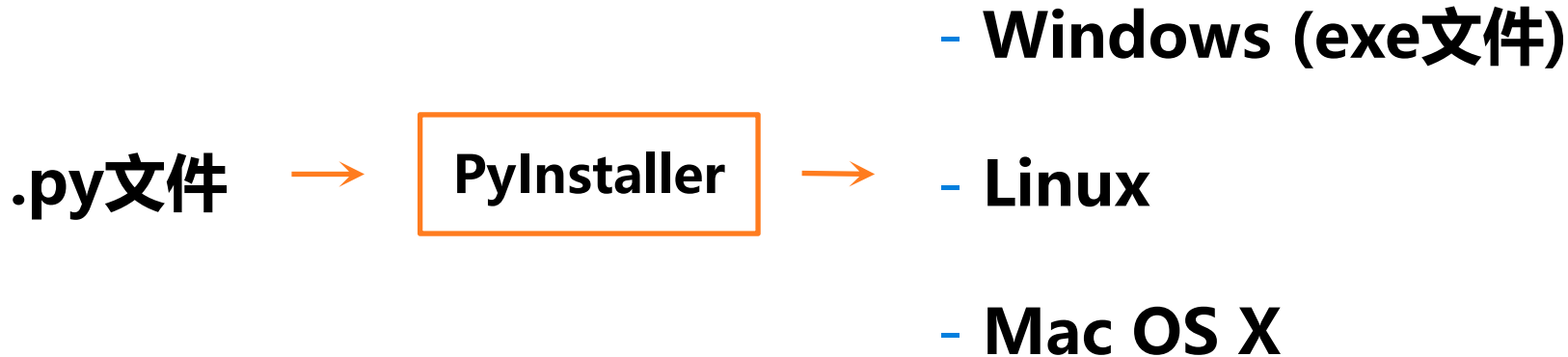
分形几何千千万

- 康托尔集、谢尔宾斯基三角形、门格海绵...
- 龙形曲线、空间填充曲线、科赫曲线...
- 函数递归的深入应用...

5.7 PyInstaller库和程序打包

嵩 天
北京理工大学

将.py源代码转换成无需源代码的可执行文件





PyInstaller库是第三方库

- 官方网站: <http://www.pyinstaller.org>
- 第三方库: 使用前需要额外安装
- 安装第三方库需要使用pip工具

PyInstaller库的安装



(cmd命令行) `pip install pyinstaller`

命令提示符 - pip install pyinstaller

```
C:\Users\Tian Song>pip install pyinstaller
```

```
Collecting pyinstaller
```

```
  Downloading PyInstaller-3.3.1.tar.gz (3.5MB)
```

Progress	Downloaded	Size	Speed	ETA
0%	10kB	93kB/s	eta 0:0	
0%	20kB	65kB/s	eta 0:	
0%	30kB	78kB/s	eta 0:	
1%	40kB	49kB/s	eta 0:	
1%	51kB	61kB/s	eta 0:	
1%	61kB	74kB/s	eta 0:	
2%	71kB	86kB/s	eta 0:	
2%	81kB	98kB/s	eta 0:	
2%	92kB	111kB/s	eta 0:	
2%	102kB	94kB/s	eta 0:	
3%	112kB	104kB/s	eta 0:	

命令提示符

```
99% |████████████████████████████████████████████████████████████████████████████████|
100% |████████████████████████████████████████████████████████████████████████████████|
████████████████████████████████████████████████████████████████████████████████ 8.3MB 11kB/s
Installing collected packages: pefile, altgraph, macholib, pywin32, pypiwin32, pyinstaller
  Running setup.py install for pefile ... done
  Running setup.py install for pyinstaller ... done
Successfully installed altgraph-0.15 macholib-1.9 pefile-2017.11.5 pyinstaller-3.3.1 pypiwin32-223 pywin32-223
```

```
C:\Users\Tian Song>
```

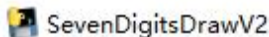
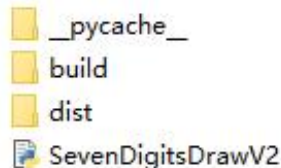

简单的使用



(cmd命令行) `pyinstaller -F <文件名.py>`

命令提示符 - pyinstaller -F SevenDigitsDrawV2.py

```
D:\PYECourse>pyinstaller -F SevenDigitsDrawV2.py
140 INFO: PyInstaller: 3.3.1
140 INFO: Python: 3.6.4
140 INFO: Platform: Windows-10-10.0.15063-SP0
156 INFO: wrote D:\PYECourse\SevenDigitsDrawV2.spec
156 INFO: UPX is not available.
172 INFO: Extending PYTHONPATH with paths
['D:\\PYECourse', 'D:\\PYECourse']
172 INFO: checking Analysis
172 INFO: Building Analysis because out00-Analysis.toc is non
existent
172 INFO: Initializing module dependency graph...
172 INFO: Initializing module graph hooks...
172 INFO: Analyzing base_library.zip ...
```



PyInstaller库常用参数

参数	描述
-h	查看帮助
--clean	清理打包过程中的临时文件
-D, --onedir	默认值, 生成dist文件夹
-F, --onefile	在dist文件夹中只生成独立的打包文件
-i <图标文件名.ico>	指定打包程序使用的图标(icon)文件

使用举例



`pyinstaller -i curve.ico -F SevenDigitsDrawV2.py`

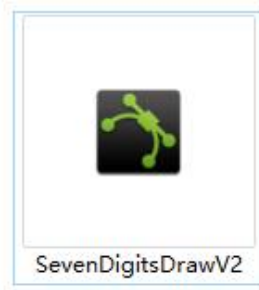


+



SevenDigitsDrawV2

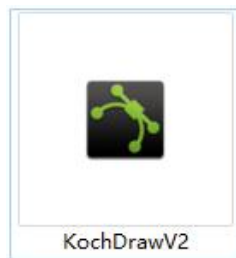
=



打包"科赫曲线绘制"

```
命令提示符 - pyinstaller -i curve.ico -F KochDrawV2.py

D:\PYECourse>pyinstaller -i curve.ico -F KochDrawV2.py
62 INFO: PyInstaller: 3.3.1
62 INFO: Python: 3.6.4
62 INFO: Platform: Windows-10-10.0.15063-SP0
62 INFO: wrote D:\PYECourse\KochDrawV2.spec
62 INFO: UPX is not available.
62 INFO: Extending PYTHONPATH with paths
['D:\\PYECourse', 'D:\\PYECourse']
62 INFO: checking Analysis
62 INFO: Building Analysis because out00-Analysis.toc is non
existent
62 INFO: Initializing module dependency graph...
62 INFO: Initializing module graph hooks...
62 INFO: Analyzing base_library.zip ...
```





单元小结: PyInstaller库和程序打包

- 使用PyInstaller库将Python程序打包为独立可执行文件
- 安装PyInstaller库、使用 `pyinstaller -F <文件名.py>`



本章小结



函数定义与调用

- 使用保留字`def`定义函数，`lambda`定义匿名函数
- 可选参数(赋初值)、可变参数(*b)、名称传递
- 保留字`return`可以返回任意多个结果
- 按位置顺序传递参数，按参数名称传递参数
- 保留字`global`声明使用全局变量，一些隐式规则



递归函数

- 函数递归的2个特征：基例和链条
- 函数递归的实现：函数 + 分支结构
- 递归典型实例：字符串反转、斐波那契数列、汉诺塔



代码复用和模块化设计

- 通过抽象赋予代码含义：函数、对象
- 模块化设计：松耦合、紧耦合



本章学习资源

配套教材：高等教育出版社《Python语言程序设计基础（第3版）》

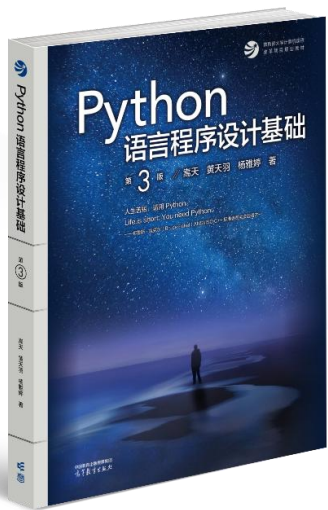
嵩天、黄天羽、杨雅婷 著



教材介绍

本书提出了以“理解和运用计算生态”为目标的Python语言教学思想，系统讲解“Python基础语法”知识体系、“Python计算方法”能力框架以及11个优秀功能库，向初学Python语言的读者展示了全新的编程语言学习路径，同时，本书在全国首次讲解大模型辅助编程的方法。

全书一共设计了18个非常具有现代感的实例，如蟒蛇绘制、天天向上的力量、七段数码管绘制、《三国演义》人物出场统计、中央一号文件词云等，绝大多数实例为作者原创，将随着内容深入不断激发读者学习Python语言的热情，因为“编程是件很有趣的事儿”。



配套微视频资源：第5章 函数和代码复用

函数的理解和定义



函数的返回值



lambda()函数



函数的使用及调用过程



函数的参数传递



局部变量和全局变量



函数递归的理解



函数递归的调用过程



函数递归实例解析



代码复用与模块化设计



PyInstaller库介绍



PyInstaller库使用



七段数码管绘制
实例讲解（上）



七段数码管绘制
实例讲解（下）



七段数码管绘制
举一反三



科赫曲线绘制
问题分析



科赫曲线绘制
实例讲解（上）



科赫曲线绘制
实例讲解（下）

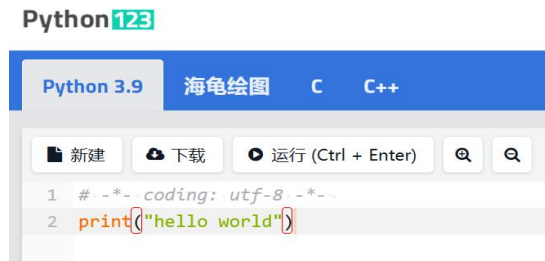


科赫曲线绘制
举一反三



在线实践平台：从 Python123 到 Learnverse

Python123
《编程》更简单



<https://python123.io>

智能升级

UPGRADE



Hi~ 欢迎来到

领航星 学习实践平台

为您提供大计算、大智能时代的全新学习体验

```
1 print("Hello World!")
2 print("Hello New World!")
3 print("Hello Python World!")
4 print("Hello New Python World!")
5
```



复制代码



领航星 Learnverse

<https://www.learnverse.cn/>

在线优质课程：Python语言程序设计



国家精品在线开放课程

(请选择最新学期)



中国大学MOOC

下载手机APP